GITHUB_WORKFLOW.md: Building This Project with Antigravity and Keeping GitHub in Sync

Put this file in your project root with `PRD.md` and `TECH_STACK.md`. It explains, step by step, how Git and GitHub fit around the code that Antigravity writes.

1. The Mental Model (read this first)

Three things are involved, and each has one job:

Thing	Job
Your project folder (`p2p-transfer/`)	Holds all the files. This folder is the **repository**.
Git (on your laptop)	Records snapshots of that folder, called **commits**. Works offline.
GitHub	An online copy of your repository. You **push** commits up to it.
Antigravity's agent	Only edits files. It does not decide what gets saved to history or sent to GitHub. **You** do that.

Key rules that remove most beginner confusion:

1. **One project = one folder = one repo.** The folder that contains `PRD.md`, `TECH_STACK.md`, `apps/`, and `packages/` is the repo

root. Run `git init` once, in that folder, and never inside `apps/web` or any subfolder.
2. **Git does not care what language the code is.** It just tracks files. Whether Antigravity writes TypeScript or you write Java elsewhere, the workflow is identical.
3. **A commit is a checkpoint, not a finish line.** Commit often, in small pieces, each with a clear message.

2. One-Time Setup (Windows 11)

2.1 Install and configure Git

1. Install **Git for Windows** from git-scm.com (the default options are fine; it includes Git Credential Manager for login).
2. Open PowerShell and set your identity:

```
git config --global user.name "Madhav"
git config --global user.email "YOUR_GITHUB_NOREPLY_EMAIL
git config --global init.defaultBranch main
```

Use your **GitHub noreply email** (GitHub → Settings → Emails → "Keep my email addresses private", then copy the `...@users.noreply.github.com` address). That way your real email does not appear in public commits.

2.2 Create the empty repository on GitHub

1. On github.com click **New repository**.
2. Name: `p2p-transfer` (you can rename later without breaking anything).
3. Description: *End-to-end encrypted peer-to-peer file and text transfer in the browser (WebRTC, Web Crypto).*
4. **Public**.
5. **Do NOT** tick "Add a README", ".gitignore", or "license". The repo must start empty so your first push has no conflicts. (You will add

a README and license from your own folder.)
6. Click **Create repository**. Copy the URL: `https://github.com/Madhav2607/p2p-transfer.git`

2.3 Prepare the folder

1. Create the folder, for example `C:\Projects\p2p-transfer`.
2. Paste these files into it: `PRD.md`, `TECH_STACK.md`, `GITHUB_WORKFLOW.md`, `.gitignore`, `.gitattributes`.
    - Windows tip: File Explorer will not let you *rename* a file to start with a dot. If `.gitignore` or `.gitattributes` arrive without the dot, rename them in PowerShell, for example `ren gitignore.txt .gitignore`.
3. Open the folder in Antigravity: **File → Open Folder** and choose `p2p-transfer`. Antigravity is built on the VS Code editor, so the **Source Control** icon in the left sidebar shows your changed files and diffs.

2.4 First commit and first push

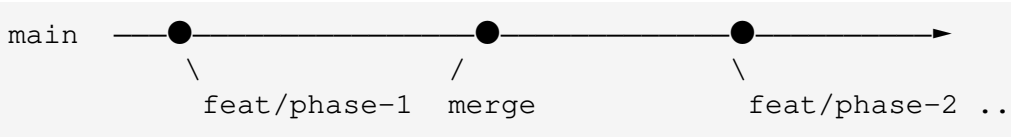
Run these in a terminal **inside the project folder** (the terminal in Antigravity opens there):

```
git init -b main
git add .
git status
git commit -m "docs: add PRD, tech stack, and git workflo
git remote add origin https://github.com/Madhav2607/p2p-t
git push -u origin main
```

- `git status` before committing should list only your docs and dotfiles. If you ever see `node_modules`, stop: your `.gitignore` is missing or misnamed.
- On the first push, a browser window opens to log in to GitHub. That is normal.
- Refresh the GitHub page: your files should be there. Setup is done.

3. The Working Loop (repeat for every phase)

`TECH_STACK.md` Section 10 splits the build into phases. Treat **each phase as one branch**.



For each phase:

1. **Checkpoint first.** Make sure `git status` is clean (everything committed). This gives you a safe point to return to if the agent makes a mess.
2. **Create the branch:**

   ```
   git switch -c feat/phase-2-signaling
   ```

3. **Give the agent exactly one phase** (see the prompt in Section 6).
4. **Run it yourself.** Start the app or run the tests (`pnpm test`, `pnpm dev`). If it does not work, ask the agent to fix it before committing.
5. **Review the diff** in the Source Control panel. Open every changed file and read it. Do not commit code you do not understand (see Section 7).
6. **Commit in small logical pieces**, not one giant commit:

   ```
   git add apps/signaling packages/protocol
   git commit -m "feat(signaling): add session store wit
   git add apps/signaling
   git commit -m "test(signaling): cover expiry and rate
   ```

7. **Push the branch:**

   ```
   git push -u origin feat/phase-2-signaling
   ```

8. **Open a Pull Request** on GitHub from that branch into `main`, even though you are solo. Write 3 to 5 lines: what changed, how you tested it. Once CI is green (after Phase 1 adds it), click **Merge**. This gives your repo the same history recruiters see in real teams.
9. **Update your laptop and tag the milestone:**

```
git switch main
git pull
git tag -a v0.2.0 -m "Phase 2: signaling server"
git push --tags
```

10. Delete the merged branch (GitHub offers a button), then start the next phase.

Phase to branch to tag map

Phase (from TECH_STACK.md)	Branch	Tag after merge
1. Scaffold	`chore/phase-1-scaffold`	`v0.1.0`
2. Signaling server	`feat/phase-2-signaling`	`v0.2.0`
3. Connect two tabs	`feat/phase-3-webrtc-connect`	`v0.3.0`
4. File transfer	`feat/phase-4-file-transfer`	`v0.4.0`
5. Crypto layer	`feat/phase-5-encryption`	`v0.5.0`
6. QR pairing	`feat/phase-6-qr-pairing`	`v0.6.0`
7. PWA and polish	`feat/phase-7-pwa-polish`	`v0.7.0`
8. P1 items	one branch per item, e.g. `feat/multi-file-queue`	`v0.8.0`

9. Docs and launch	`docs/phase-9-launch`	`v1.0.0`

4. Commit Message Convention

Use **Conventional Commits**: `type(scope): short description in present tense`.

Type	Use for
`feat`	A new capability
`fix`	A bug fix
`test`	Adding or changing tests
`docs`	README, PRD, comments
`refactor`	Code change with no behavior change
`chore`	Tooling, config, dependencies
`ci`	GitHub Actions changes

Good: `feat(web): stream received chunks to disk with File System Access API` Bad: `update`, `fix stuff`, `final version`

5. Rules for Working with the Agent

The agent is fast but it can break things silently. These habits protect your repo:

1. **You own Git.** The agent edits files; you stage, commit, push, and merge. Tell it so in your prompts.
2. **Never allow** the agent to run `git push`, `git push --force`, `git reset --hard`, `git clean -fd`, or `git checkout .` without asking you. In Antigravity's settings, keep terminal command execution on a mode that asks for approval (the setting

name differs between versions, so look for terminal or command approval options).
3. **Commit before every agent task** so a bad run can be undone with `git restore .` or by switching back to your last commit.
4. **One phase per task.** Large vague prompts produce large unreviewable diffs.
5. **Secrets never go in Git.** Environment values (TURN credentials, host config) live in a `.env` file that is git-ignored. Commit a `.env.example` with fake placeholder values instead. If you ever commit a real secret, rotate it immediately; deleting the file later does not remove it from history.
6. **Commit the lockfile** (`pnpm-lock.yaml`). Do not commit `node_modules`, `dist`, or `.env`.
7. **Ask the agent to explain.** After each phase, ask: "Explain how this phase works and what could go wrong." Keep the answer for your README notes.

6. Prompt Block to Paste at the Start of Every Antigravity Session

```
Project rules for this session:
- Read PRD.md and TECH_STACK.md before changing anything.
- Work on ONE phase only: <PHASE NAME FROM TECH_STACK.md
- Do not run git push, git reset --hard, git clean, or fo
- Do not stage or commit unless I ask. When done, list th
  suggest Conventional Commit messages I can use.
- Never put secrets in code. Use environment variables an
- When finished, tell me exactly how to run and test what
```

7. Make the Repo Work as a Portfolio Piece

Recruiters and interviewers spend under a minute on a repo. Make that minute count:

- **README.md** at the root with: one-line pitch, a demo GIF (record two windows transferring a file), a feature list, the architecture diagram from `PRD.md`, how to run it locally, the threat model summary, and a link to the live demo.
- **Repo settings:** add the description, a website link (live demo), and topics such as `webrtc`, `end-to-end-encryption`, `typescript`, `p2p`, `file-transfer`, `pwa`.
- **Pin the repo** on your GitHub profile (profile → Customize your pins).
- **LICENSE:** add an MIT license (GitHub: Add file → Create new file → type `LICENSE` → "Choose a license template").
- **GitHub Actions badge** in the README once CI is running.
- **Issues as a roadmap:** create one Issue per P1 item from the PRD and label them. Close them from commits with `Fixes #3`.
- **Releases:** turn your tags (`v0.4.0` etc.) into GitHub Releases with two lines of notes each. A steady release history shows consistent progress.
- **Resume line:** link `github.com/Madhav2607/p2p-transfer` and use the one-liner from Section 16 of `PRD.md`.

Be honest and be able to defend the code

State in the README that you built this with AI assistance (Antigravity) and that you designed the architecture and reviewed the code. This is normal and increasingly expected. But the repo only helps you if **you can explain every part of it**. Interviewers will pick a file and ask why. So, for each phase, before merging:

- Be able to explain the data flow in your own words (who sends what to whom).
- Be able to explain *why* each security choice exists (why DTLS fingerprints are bound into the key, why nonces must never repeat).
- Modify something small by hand (rename a variable, change a timeout, add a test) so you have touched the code yourself.

A smaller project you fully understand beats a bigger one you cannot explain.

8. Troubleshooting

Problem	Fix
`node_modules` shows up in `git status`	`.gitignore` is missing or misnamed. Fix it, then run `git rm -r --cached node_modules` and commit.
`LF will be replaced by CRLF` warnings	Harmless on Windows. The included `.gitattributes` keeps line endings consistent.
`! [rejected] ... non-fast-forward` on push	GitHub has commits you do not have (for example you edited on the website). Run `git pull --rebase`, then push again.
Pushed something secret	Rotate the secret first. Then ask for help cleaning history; do not just delete the file.
Agent made a mess, nothing committed	`git restore .` throws away uncommitted changes to tracked files. Use `git status` first to check what you would lose.
Agent made a mess, you already committed	`git log --oneline` to find the good commit, then `git switch -c rescue <hash>` to branch from it.
Push asks for a password and fails	GitHub no longer accepts account passwords. Let Git Credential Manager open the browser login, or run `gh auth login` if you installed GitHub CLI.
Not sure what state you are in	`git status` and `git log --oneline -10` answer 90% of questions.

9. Command Cheat Sheet

```
git status                          # what changed
```

```
git diff                          # see changes line by lin
git add <path>                    # stage specific files
git commit -m "feat(x): msg"      # save a checkpoint
git switch -c <branch>            # new branch and switch t
git switch main                   # go back to main
git push -u origin <branch>       # first push of a branch
git push                          # later pushes
git pull                          # get latest from GitHub
git log --oneline -10             # recent history
git restore .                     # discard uncommitted cha
git tag -a v0.2.0 -m "msg"        # mark a milestone
git push --tags                   # upload tags
```